UNIVERSIDAD POLITECNICA SALESIANA - SEDE QUITO

ASIGNATURA: Sistemas Distribuidos

DOCENTE: Julio Proaño Ricardo Orellana

CARRERA: Computación

GRUPO: 1

INTEGRANTES:

Jairo Alejandro Ojeda Herrera

Adrián Fernando Barriga Castillo

FECHA: 04-11-2026

Tabla de contenido

Práctica Guiada: Programación Distribuida y Llamadas a Procedimientos Remotos (RPC) con Pyro4 en Python 3

1. Preguntas de Investigación y Fundamentación Teórica 3

¿Cuál es el rol del Name Server en Pyro4 y por qué es preferible al direccionamiento estático por URI? 3

2. Explique la diferencia entre las políticas de instancia @Pyro4.behavior(instance_mode="session") y instance_mode="percall". 3

3. ¿Qué mecanismos de serialización utiliza Pyro4 por defecto y qué implicaciones de seguridad existen al intercambiar objetos complejos? 4

4. Investigue cómo manejar la reconexión automática de clientes cuando el servidor se desconecta temporalmente 4

5. Actividades Prácticas Dirigidas 4

Nivel 1: Calculadora Remota Básica (Fundamentos de RPC) 4

Nivel 2: Sistema de Notificaciones con Name Server 5

Nivel 3: Banco Distribuido y Control de Estado (Stateful Services) 6

Nivel 4: Procesamiento Concurrente y Callback Asíncrono 7

6. Conclusiones y Lecciones Aprendidas 8

Link de GitHub: https://github.com/Alejandro120325/Pr-ctica-de-Laboratorio-RPC-con-Python.git

Práctica Guiada: Programación Distribuida y Llamadas a Procedimientos Remotos (RPC) con Pyro4 en Python

1. **Preguntas de Investigación y Fundamentación Teórica**

   ¿Cuál es el rol del Name Server en Pyro4 y por qué es preferible al direccionamiento estático por URI?

El Name Server (Servidor de Nombres) en Pyro4 actúa como un directorio o registro centralizado donde los servidores publican sus objetos remotos bajo un nombre lógico (ej. "practica.mensajeria"). Es preferible al direccionamiento estático porque desacopla la ubicación física del servicio de su identificación. Con direccionamiento estático por URI (Nivel 1), si el servidor cambia de IP o puerto, todos los clientes deben actualizar su código con la nueva URI. Con el Name Server, el cliente solo pide conectarse al nombre lógico, y el Name Server se encarga de redirigirlo a la ubicación actual, facilitando la escalabilidad y tolerancia a fallos.

2. **Explique la diferencia entre las políticas de instancia @Pyro4.behavior(instance_mode="session") y instance_mode="percall".**

- **instance_mode="session":** Mantiene el estado. El servidor crea una instancia única del objeto para cada cliente (proxy) conectado. Mientras el cliente mantenga la conexión (sesión), las variables de la instancia persisten, como se demostró en el Nivel 3 con el saldo bancario independiente para cada usuario.
- **instance_mode="percall":** No mantiene estado (stateless). El servidor crea una instancia nueva y limpia del objeto en cada llamada a un método, destruyéndola al terminar. Si se usara en el banco, el saldo volvería a cero en cada operación.

3. **¿Qué mecanismos de serialización utiliza Pyro4 por defecto y qué implicaciones de seguridad existen al intercambiar objetos complejos?**

Pyro4 utiliza por defecto el serializador "serpent", que es seguro y convierte los datos a tipos primitivos estándar (diccionarios, listas, strings, números). Anteriormente se utilizaba "pickle", pero este permitía la ejecución de código arbitrario al deserializar objetos complejos, lo que representaba una vulnerabilidad crítica de seguridad (RCE - Ejecución Remota de Código). Al usar serpent, Pyro4 restringe el intercambio a datos puros, impidiendo que un atacante envíe una clase maliciosa para ser ejecutada en el servidor.

4. **Investigue cómo manejar la reconexión automática de clientes cuando el servidor se desconecta temporalmente.**

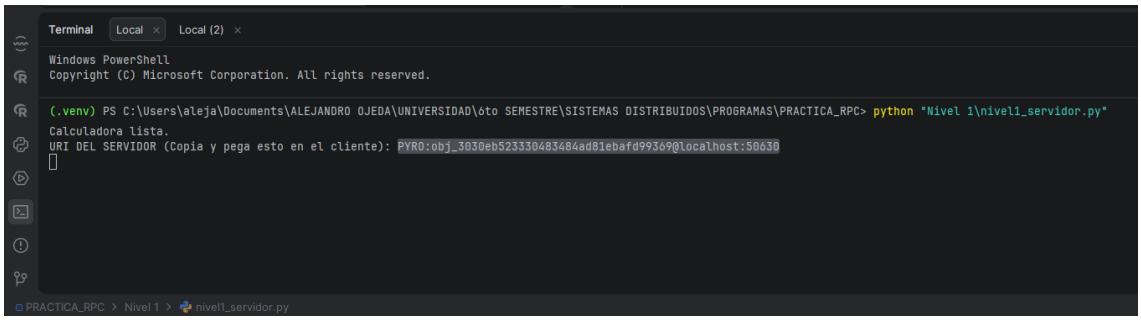
Para manejar la reconexión automática en Pyro4 (manejo de fallas de red), se debe interceptar la excepción Pyro4.errors.ConnectionClosedError o Pyro4.errors.CommunicationError durante la llamada remota. Al detectarla, el cliente puede implementar un bucle con un mecanismo de reintento (retry) que utilice Pyro4.Proxy.pyroReconnect() (en versiones antiguas) o que simplemente vuelva a instanciar el Proxy consultando nuevamente al Name Server tras un breve tiempo de espera (sleep), hasta que el servidor vuelva a estar disponible.

5. **Actividades Prácticas Dirigidas**

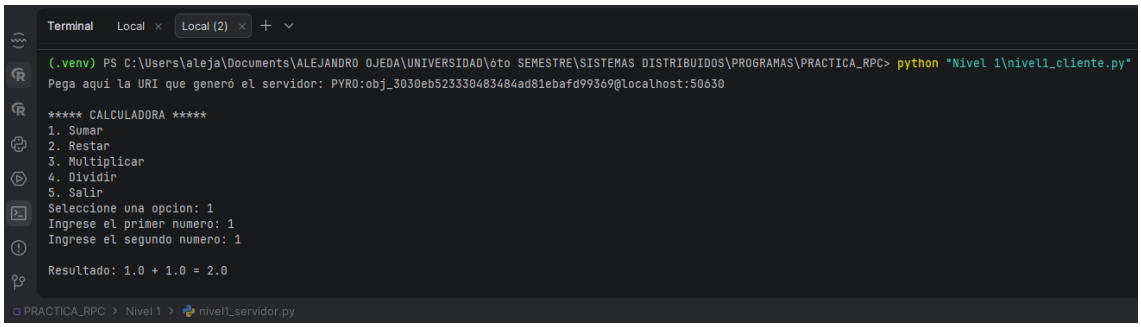
Nivel 1: Calculadora Remota Básica (Fundamentos de RPC)

En este nivel se implementó la invocación a métodos remotos mediante direccionamiento directo. El servidor publicó el objeto Calculadora generando una URI única, la cual el cliente debió ingresar manualmente para establecer la conexión, sin mediación de un Servidor de Nombres.

Servidor

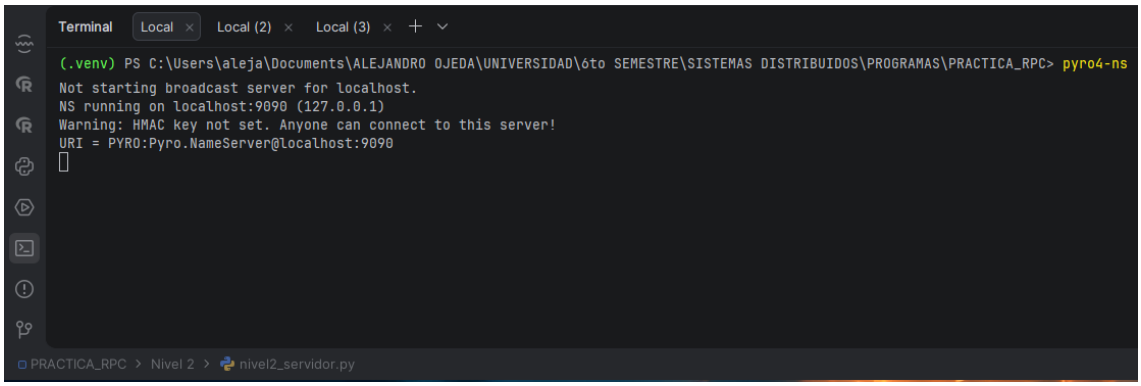


Cliente

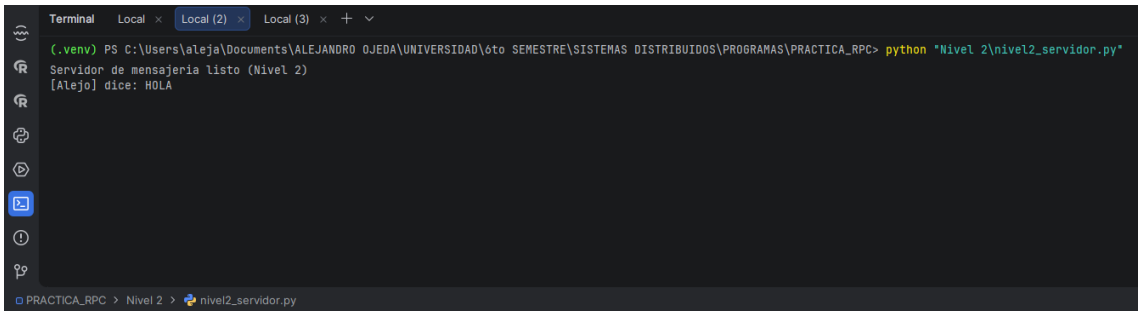


Nivel 2: Sistema de Notificaciones con Name Server

Se desacopló la ubicación del servidor utilizando el Name Server de Pyro4. El servidor se registró bajo el identificador lógico "practica.mensajeria", permitiendo al cliente localizarlo dinámicamente mediante Pyro4.locateNS() e iniciar el intercambio de mensajes.



Servidor

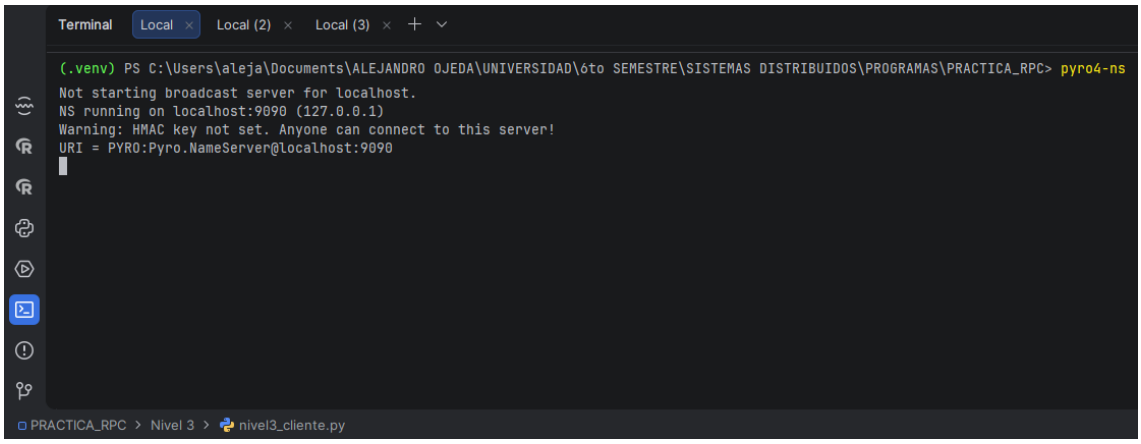


Cliente



Nivel 3: Banco Distribuido y Control de Estado (Stateful Services)

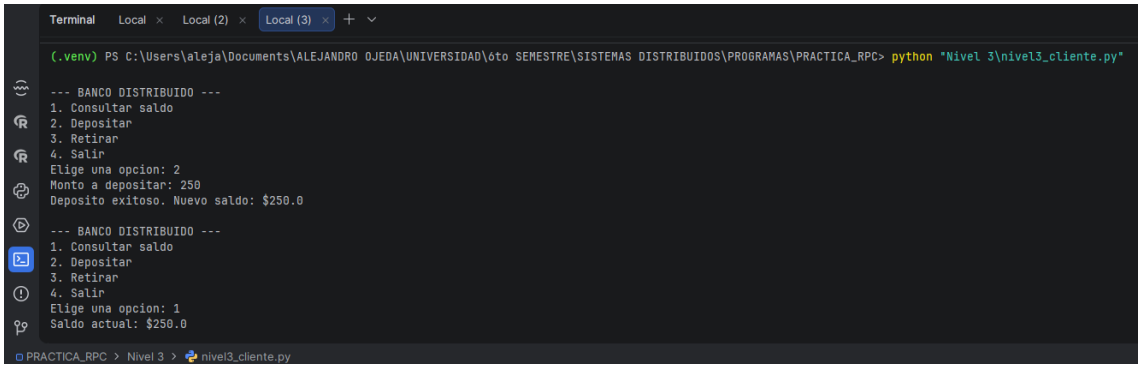
Se demostró la gestión del estado en sistemas distribuidos mediante el decorador @Pyro4.behavior(instance_mode="session"). Esto permitió que el servidor mantuviera una instancia de CuentaBancaria en memoria por cada cliente conectado, garantizando que los depósitos y retiros persistieran en el saldo local del usuario durante su sesión.



Servidor

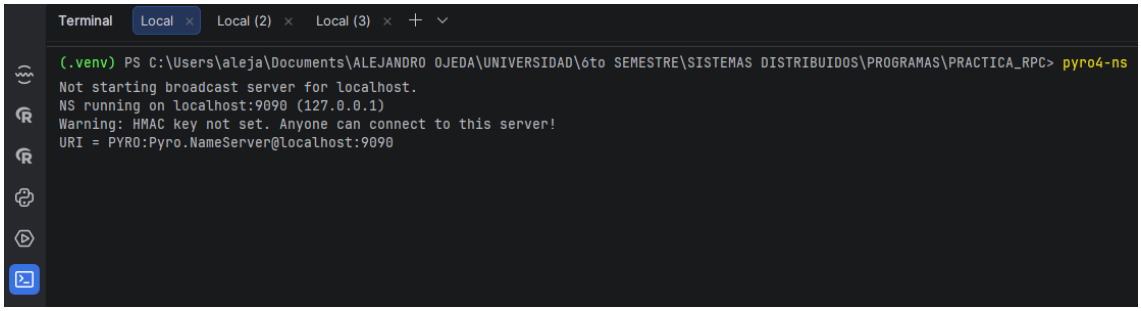


Cliente



Nivel 4: Procesamiento Concurrente y Callback Asíncrono

Para evitar bloqueos en el cliente durante operaciones pesadas, se implementó el uso de objetos asíncronos (Pyro4.Future). Mientras el servidor procesaba una tarea simulada de 10 segundos, el hilo principal del cliente continuó ejecutándose libremente, consultando el estado del future (future_result.ready) y recuperando el valor final al completarse.



Servidor



Cliente



6. Conclusiones y Lecciones Aprendidas

El desarrollo de esta práctica permitió comprender a profundidad el ciclo de vida de un objeto remoto en Python utilizando Pyro4. A diferencia de las arquitecturas basadas en REST APIs, que operan bajo un esquema estrictamente sin estado (stateless) basado en los métodos de HTTP y formato JSON, Pyro4 ofrece la ventaja de manipular objetos de Python nativos a través de la red, facilitando la persistencia de sesión transparente (Nivel 3). Sin embargo, frente a soluciones modernas como gRPC, Pyro4 puede presentar limitaciones en ecosistemas políglotas (es fuertemente dependiente de Python) y en el control estricto de tipos de datos que gRPC soluciona mediante Protocol Buffers.